

Nima Memarzadeh

Nathan Braun

CSD380

06/25/2025

Case Study — Strangler Pattern at Blackboard Learn (2011)

The Strangler Fig Pattern has emerged as a safe, gradual path to modernizing legacy monoliths into modular, scalable microservices. This case study analyzes how Blackboard Learn, an educational software platform, applied the Strangler Fig Pattern to evolve beyond the limitations of its legacy codebase in 2011. It also draws comparisons from Shopify and Thoughtworks' more recent applications of the same pattern.

Summary of the Blackboard Learn Case

By 2011, Blackboard Learn had become a large J2EE codebase full of legacy Perl scripts dating back to 1997 (Kim et al., 2016). Since the codebase was highly coupled, it took days or weeks to get changes through from development to integration testing. The integration teams sometimes had to wait 36 hours to get feedback on the changes they had made, which significantly decreased productivity and increased system fragility.

As the product grew, these problems became more apparent. Figure 13.2 shows that, around 2010, the number of code commits started to decrease, even as the total lines of code continued to increase, which indicated that the system was getting harder to work with. According to Ashman, Blackboard's chief architect, "To me, it said we needed to do something, otherwise the problems would just get worse, with no end in sight" (Kim et al., 2016).

In 2012, the team embraced the Strangler Fig Pattern. The new, re-architected system included Building Blocks—modular pieces connected by stable APIs. This allowed:

- Higher developer autonomy
- Faster iteration and quicker feedback
- Safer deployments (because failures were localized instead of being system-wide)

Figure 13.3 shows that both code commits and lines of code started to grow exponentially after the transition. Developers also reported higher quality and productivity due to increased modularity.

Comparative Insights: Shopify’s Strangler Fig Implementation

In 2019, Shopify faced the same challenge as Blackboard Learn had in 2011, but for its largest God Object, the Shop model, weighing in at over 3000 lines of code. Rather than rewriting it, Shopify took a series of incremental steps, following a seven-step process. They defined a new interface, wired up to a new data source, backfilled the old data, and finally deprecated the legacy code (Chang, 2020). Key enablers were strong test coverage, version control, and CI/CD automation. It ensured safe steps and iterative development were possible.

Strategic Considerations: Thoughtworks’ Modernization Framework

Thoughtworks provides additional context for the technical, organizational, and procedural aspects of the Strangler Fig Pattern. From a technical perspective, test-first parity suites, canary releases, dark launches, and observability tools play a crucial role in ensuring a safe transition (Chandrasekaran, 2023a). They also emphasize the importance of minimizing two-way data synchronization and therefore the importance of picking thin slices for migration. In one case study, Thoughtworks helped a grocery retailer modernize a slow and unmanageable coupon system. By picking the `/get_coupons` endpoint as the first “slice”, they implemented a high-quality replacement that ran quickly and with no data synchronization. Meanwhile, the

legacy system remained live through an API gateway (Chandrasekaran, 2023b). The two were running in parallel until consumer acceptance testing was complete.

Lessons Learned

1. Breaking up a monolith into smaller components can improve agility.
2. Developers can iterate more quickly and safely using a small, independent API to get things done.
3. The Strangler pattern allowed us to move incrementally instead of forcing a monolithic rewrite.
4. If you allow the various Building Blocks to be self-contained and self-hosted, they are able to be delivered incrementally without impacting other components of the monolith.
5. You can measure things like lines of code written per developer or commit frequency to identify issues and measure success.
6. You can still keep the old monolith in place, but move away from it by rewriting the new work into a different code base.

The Blackboard Learn case is an excellent illustration of how the Strangler Fig Pattern can be used to modernize a legacy system without interrupting business operations. With a focus on modularization, API design, and incremental change, Blackboard was able to revitalize its architecture and software development practices. This case also shows how technical strategy, informed by appropriate delivery practices, can enable agility, speed, and resiliency in large software systems.

Works Cited

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations*. IT Revolution Press.

Chang, A. (2020). *Refactoring Legacy Code with the Strangler Fig Pattern*. Shopify Engineering Blog. Retrieved from <https://shopify.engineering/refactoring-legacy-code-strangler-fig-pattern>

Chandrasekaran, P. (2023a). *Embracing the Strangler Fig pattern for legacy modernization: Part Three – Overcoming technology challenges*. Thoughtworks. Retrieved from <https://www.thoughtworks.com/en-us/insights/articles/embracing-strangler-fig-pattern-legacy-modernization-part-three>

Chandrasekaran, P. (2023b). *Embracing the Strangler Fig Pattern for Legacy Modernization: Part One*. Thoughtworks. Retrieved from <https://www.thoughtworks.com/en-us/insights/articles/embracing-strangler-fig-pattern-legacy-modernization-part-one>